

RECOGNITION SYSTEM FOR INDIAN SIGN LANGUAGE USING ORB WITH CNN AND SVM

*Report to be submitted to the SASTRA Deemed to be University
as the requirement for the course*

CSE300 - MINI PROJECT

Submitted by

RAMALAKSHMI B

(Reg. No: 124003241, B. Tech - CSE)

SOWMYA VARSHINI M

(Reg. No: 124003304, B. Tech - CSE)

THRISHA K

(Reg. No: 124003344, B. Tech - CSE)

MAY 2023



SCHOOL OF COMPUTING

THANJAVUR, TAMIL NADU, INDIA – 613 401



SCHOOL OF COMPUTING

THANJAVUR – 613 401

Bonafide Certificate

This is to certify that the report titled “**Recognition System for Indian Sign Language using orb with cnn and svm**” submitted as a requirement for the course, CSE300: **MINI PROJECT** for B.Tech. is a bonafide record of the work done by **Ms. RAMALAKSHMI B (Reg. No: 124003241, CSE), Ms. SOWMYA VARSHINI M (Reg. No: 124003304 CSE), Ms. THRISHA K (Reg. No: 124003344, CSE)** during the academic year 2022-23, in the School of Computing, under my supervision.

Signature of Project Supervisor :

Name with Affiliation

: Dr. P. Manivannan, Assistant Professor - III

Date

: 16-May-23

Mini Project *Viva Voce* held on _____

Examiner 1

Examiner 2

Acknowledgements

We would like to thank our Honourable Chancellor Prof. R. Sethuraman for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honourable Vice-Chancellor **Dr. S. Vaidhyasubramaniam** and **Dr. S. Swaminathan**, Dean, Planning & Development, for the encouragement and strategic support at every step of our college life.

We extend our sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

We extend our heartfelt thanks to **Dr. A. Umamakeswari**, Dean, School of Computing,

Dr. S. Gopalakrishnan, Associate Dean, Department of Computer Application, **Dr. B.Santhi**, Associate Dean, Research, **Dr. V. S. Shankar Sriram**, Associate Dean, Department of Computer Science and Engineering, **Dr. R. Muthaiah**, Associate Dean, Department of Information Technology and Information & Communication Technology

Our guide **Dr. P. Manivannan**, Assistant Professor – III, School of Computing was the driving force behind this whole idea from the start. His deep insight in the field and invaluable suggestions helped us in making progress throughout our project work. We also thank the project review panel members for their valuable comments and insights which made this project better.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Computing who have either directly or indirectly helped us in the completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. We thank you all for providing me an opportunity to showcase my skills through project.

List of Figures

Figure No.	Title	Page No.
Fig 1.1	CNN Architecture	3
Fig 1.2	Support Vector Machine	4
Fig 4.1	Preprocessed Images	21
Fig 4.2	Feature Extraction Using ORB	22
Fig 4.3	Feature Extraction Using HOG	24
Fig 4.4	Classification Using CNN	25
Fig 4.5	Model Graphs of CNN	26
Fig 4.6	Classification Using SVM	27

Abbreviations

ISL	Indian Sign Language
HGR	Hand Gesture Recognition
CNN	Convolutional Neural Network
SVM	Support Vector Machine
BOVW	Bag of Visual Words
ORB	Oriented FAST and Rotated BRIEF
HOG	Histogram of Gradients

Abstract

Hand signs are an effective form of communication that has many applications. Worldwide they are commonly used by speech impaired people. In fact, in India about one percent of the total population belong to this category. This is the key reason why it would have a huge beneficial effect on these people to incorporate a framework that would understand the Indian Sign Language. In this paper, we present a technique that uses the Bag of Visual Words model (BOVW) to recognize the Indian sign language alphabets (A-Z) and the digits (0-9) in the live video stream and output the predicted labels in the form of text. Segmentation is done based on skin color and the background subtraction. From the images ORB (Oriented FAST and Rotated BRIEF) features have been extracted and histograms are generated to map the sign with corresponding labels. The SVM (Support Vector Machine) and CNN (Convolutional Neural Networks) are used for classification.

KEYWORDS: Hand sign recognition, Indian sign language (ISL), ORB features, SVM, CNN.

Table of Contents

Title	Page.no
Bonafide Certificate	ii
Acknowledgements	iii
List of Figures	iv
Abbreviations	v
Abstract	vi
1. Summary of the base paper	1
2. Merits and Demerits of the base paper	6
3. Source Code	7
4. Snapshots	20
5. Conclusion and Future Plans	29
6. References	30

CHAPTER 1

SUMMARY OF THE BASE PAPER

BASE PAPER DETAILS

Title: Recognition system for Indian Sign Language using ORB with SVM and CNN

Journal Name: Array

Year of Publication: 2022

Paper Link: <https://www.sciencedirect.com/science/article/pii/S2590005622000121>

1.1 INTRODUCTION

Communication has always played a vital role in human life. Normal human beings do not have much difficulty interacting with each other and can express themselves easily through speech, gestures, body language, reading, writing, speech being widely used among them. However, people affected by speech impairment rely only on sign language. This implies a requirement for sign language recognizers which can recognize and convert sign language into spoken or written language. The main Objective is to provide a more efficient and effective means of communication for individuals with hearing impairments in various domains such as education, employment, and social interactions and to minimize the time and cost involved in training and testing the system by utilizing machine learning algorithms. The main aim of this project is that we use the Bag of Visual Words model (BOVW) to recognize Indian sign language alphabets (A-Z) and digits (0-9). Segmentation is done based on skin color as well as background subtraction. ORB and HOG features have been extracted from the images and SVM and CNN are used for classification. The goal is to develop an ISL recognition system that is user-friendly, accurate, and accessible to individuals with hearing impairments and equal opportunities for all.

1.2 FEATURE EXTRACTION:

1.1.1 ORB:

ORB makes use of a modified version of the FAST key point detector and BRIEF descriptor with some features to improve the performance. FAST is used to detect features from input images. FAST features are not scale-invariant and rotation invariant. ORB does not require any major computations. It is scale invariant, rotational invariant.

The ORB image matching algorithm is generally divided into three steps:

- Feature detection
- Feature description
- Feature matching

1.1.2 HOG:

HOG is a feature extraction method in computer vision that analyze the distribution of gradient orientations in an image to extract local features. The basic idea behind HOG is to divide the image into small rectangular regions called cells and compute the gradient orientation of each pixel within the cell. The gradient orientations are then binned into a histogram of gradient orientations. The histograms are normalized to reduce the effect of lighting variations and other factors that may affect the image's appearance. Finally, the histograms are concatenated to form a feature vector that represents the image.

1.2 CLASSIFICATION:

1.2.1 CNN:

Convolutional Neural Networks (CNNs) are a class of deep learning algorithms that have been widely used in computer vision applications such as image and video recognition, object detection, and image segmentation. The architecture of a CNN typically consists of several layers, including convolutional layers, pooling layers, and fully connected layers. In the convolutional layers, filters are applied to the input image to extract local features, while the pooling layers down sample the feature maps to reduce the computation and memory requirements. The fully connected layers at the end of the network perform the classification or regression tasks.

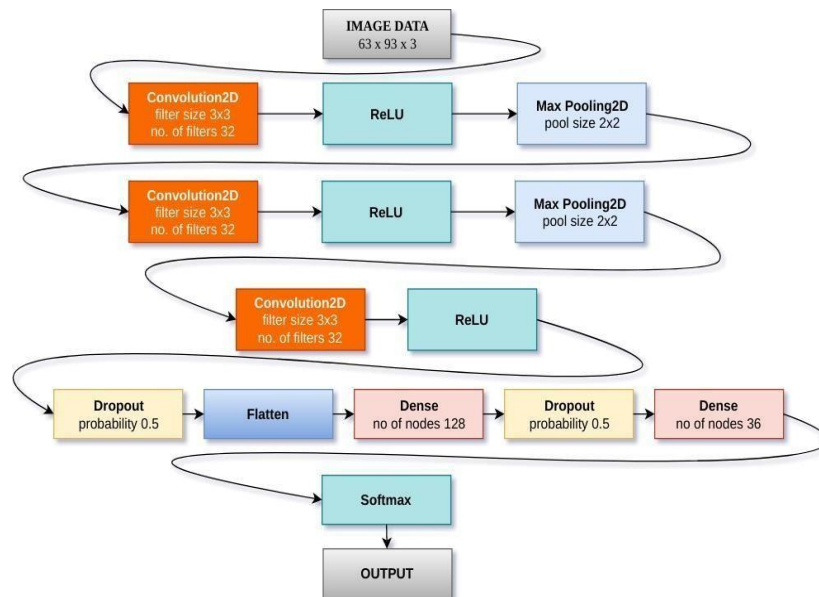


Fig 1.1 CNN Architecture

1.2.2 SVM:

It's a machine learning algorithm used for classification. The SVM algorithm works by finding the hyperplane that separates the different classes in the feature space. The key idea behind SVMs is to find the hyperplane that maximizes the margin, which is the distance between the closest points of the different classes. The points that are closest to the hyper plane are called support vectors. There are two types of SVM,

- Linear SVM
- Non-linear SVM

1.2.3 Linear SVM:

Linear SVM is a type of SVM that uses a linear kernel function to separate the data points in a high-dimensional space. The goal of linear SVM is to find a hyperplane that separates the data points into different classes while maximizing the margin between the two classes.

It is particularly useful in situations where the data is linearly separable, which means that it is possible to separate the data points using a straight line, plane, or hyperplane. It is also computationally efficient, which means that it can handle large datasets with many features.

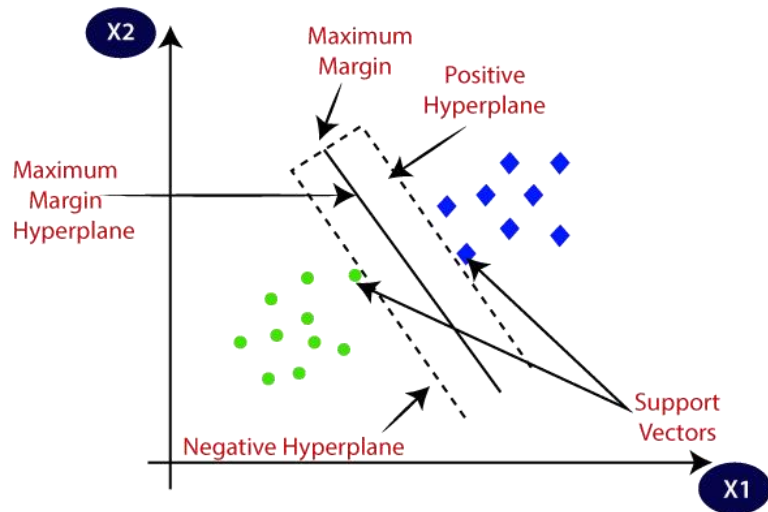


Fig 1.2 Support Vector Machine

1.3 RESULT:

Here the dataset is divided into two sets. The training set consists of 70% of the total data and the testing set consists of 30% of the remaining data. Both the classifiers (SVM and CNN) have given high accuracy on the images, but CNN has performed better with a lesser number of features. The system is trained to recognize 36 signs (26 alphabets and 10 digits).

1.4.1 SVM Performance:

SVM has given an accuracy of 75% on the test data. The calculated values of precision and recall alphabets and digits classified using SVM shows an overall accuracy of 75%.

1.4.2 CNN Performance:

Using CNN, we have observed an overall accuracy of 99% on a training set whereas testing accuracy of 75%. The total epochs are 20. We have trained our model with a categorical cross entropy loss function and soft-max function as the activation function.

1.4.3 Real Time Testing:

To build a real-time recognition system, we have integrated the feature extraction and classification steps into a single pipeline. This pipeline takes the input from a camera or a video stream, pre-process the images, extracts features using the pre-trained CNN model and classifies the images using the SVM classifier. Being a user independent model it is also capable of solving the problem of background dependency with the condition of keeping the camera still. Our model is capable of recognizing double hand signs and translating the visual information to text.

1.4.4 Quantitative analysis:

In terms of true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN), the formula of the accuracy can be written as,

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{FN} + \text{TN}}$$

1.4.5 Performance Metrics:

Precision:

Precision is the ratio of true positive (TP) predictions to the total number of positive (TP+FP) predictions made by the model.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall:

Recall is the ratio of true positive (TP) predictions to the total number of actual positive (TP + FN) cases in the data.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

F1-Score:

F1-score is the harmonic mean of precision and recall, and is often used as a summary metric that balances the two. F1-score ranges from 0 to 1, with a higher score indicating better performance.

$$\text{F1-Score} = \frac{2 * (\text{Recall} * \text{Precision})}{\text{Recall} + \text{Precision}}$$

CHAPTER 2

MERITS AND DEMERITS OF THE BASE PAPER

The following are the merits and demerits of "Indian Sign Language Recognition System using ORB with SVM and CNN":

Merits:

1. Improved accuracy: CNN and SVM are machine learning algorithms that can accurately classify and recognize complex visual patterns.
2. Efficiency: It can process large amounts of data quickly and efficiently.
3. Effective Feature Extraction: Feature Extraction can help to reduce the dimensionality of input data, making it easier for machine learning models to identify patterns and distinguish between different signs.
4. Enhanced accessibility: By facilitating sign language communication for deaf and speech impaired people, the proposed system has the potential to improve the quality of life of people with disabilities.

Demerits:

1. Higher complexity: Proposed techniques may be more complex to implement and require more computing power than existing techniques and leading to cost.
2. Limited Comparison with existing systems: Only three existing state-of-the-art systems for ISL gesture recognition have been compared by the authors with the proposed system, which may not give a thorough assessment of the performance of the system.
3. Limited availability of training data: The availability of large, high-quality datasets of Indian Sign Language can be a limiting factor for the development of accurate and robust sign language recognition systems.

CHAPTER 3

SOURCE CODE

Preprocessing code:

```
import os
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import cv2
import imageio
from skimage.feature import hog

image_path='/content/drive/MyDrive/datasplit/Train'
output_path = '/content/drive/MyDrive/Preprocessed Images/'

def loadImages(path,label):
    image_files=sorted([os.path.join(path,label,file)
        for file in os.listdir(path+str('/')+label) if file.endswith('.jpg')
    ])
    return image_files

def display(img,title="Original"):
    plt.imshow(img,cmap='gray'),plt.title(title)
    plt.show()

def preprocess_images(data, label):
    count = 0
    for image in data:

        img = imageio.imread(image)

        gray_img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
        hsv_img = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)
        skin_color_lower = np.array([0, 40, 30], np.uint8)
        skin_color_upper = np.array([43, 255, 255], np.uint8)

        skin_mask = cv2.inRange(hsv_img, skin_color_lower, skin_color_upper)
```

```

skin_mask = cv2.medianBlur(skin_mask, 5)
skin_mask = cv2.addWeighted(skin_mask, 0.5, skin_mask, 0.5, 0.0)

kernel = np.ones ((5, 5), np.uint8)
skin_mask = cv2.morphologyEx(skin_mask, cv2.MORPH_CLOSE, kernel)
hand = cv2.bitwise_and (gray_img, gray_img, mask=skin_mask)
canny = cv2.Canny(hand, 60, 60)

hog_features = hog (canny, orientations=9, pixels_per_cell=(8, 8),
cells_per_block=(2,2), multichannel=False)

output_file = output_path + label + '_' + str(count) + '.jpg'
cv2.imwrite(output_file, canny)

count += 1

signs=['0','1','2','3','4','5','6','7','8','9','A','B','C','D','E','F','G','H','I','J','K','L','M','N','O','P','Q','R','S','T','U','V','W','X','Y','Z']
for label in signs:
    images = []
    images=loadImages(image_path,label)
    preprocess_images(images,label)

```

Feature Extraction using ORB:

```

import numpy as np
import cv2
from google.colab.patches import cv2_imshow
img = cv2.imread('/content/drive/MyDrive/Preprocessed Images/A_10.jpg')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
orb = cv2.ORB_create(nfeatures=2000)
kp = orb.detect(gray, None)
kp, des = orb.compute(gray, kp)

img1 = cv2.drawKeypoints(gray, kp, None, (0,0,255), flags = 0)
cv2_imshow(img1)
cv2.waitKey(0)
cv2.destroyAllWindows

```

Feature Extraction using HOG:

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
%matplotlib inline

from skimage.io import imread, imshow
from skimage.transform import resize
from skimage.feature import hog
from skimage import data, exposure
import os
img = imread('/content/drive/MyDrive/Preprocessed Images/C_0.jpg')
imshow(img)
print(img.shape)

resized_img = resize(img, (100,150))
imshow(resized_img)
print(resized_img.shape)

fd, hog_img = hog(resized_img, orientations=9, pixels_per_cell=(8, 8),
                  cells_per_block=(2, 2), visualize=True, multichannel=False)

fd.shape
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7), sharex=True, sharey=True)
ax1.imshow(resized_img, cmap=plt.cm.gray)
ax1.set_title('Resized image')

hog_img_rescaled = exposure.rescale_intensity(hog_img, in_range=(0, 10))

ax2.imshow(hog_img_rescaled, cmap=plt.cm.gray)
ax2.set_title('Histogram of Oriented Gradients (HOG)')

plt.show()
```

Classification using CNN:

```
import numpy as np
import cv2
import os
import matplotlib.pyplot as plt
from keras.preprocessing.image import ImageDataGenerator
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, Reshape
from skimage.feature import hog
import tensorflow as tf
import keras
from tensorflow.keras import layers

train_dir = '/content/drive/MyDrive/datasplit/Train'
val_dir = '/content/drive/MyDrive/datasplit/Test'
batch_size = 32
img_size = (64,64)
train_datagen =
    ImageDataGenerator(rescale=1./25
        5,
        rotation_range=20,
        zoom_range=0.2,
        horizontal_flip=True
    )
val_datagen = ImageDataGenerator (rescale=1./255)
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical'
)
val_generator =
    val_datagen.flow_from_directory(val_dir,
        target_size=img_size,
        batch_size=batch_size,
        class_mode='categorical'
    )
def extract_hog_features(img):
    fd, hog_image = hog(img, orientations=9, pixels_per_cell=(8, 8), cells_per_block=(2, 2
), visualize=True, multichannel=True)
    return fd
```

```

def get_data_generator(generator):
    while True:
        x_batch, y_batch = generator.next()
        x_batch_hog = np.zeros((x_batch.shape[0], 1764))

        for i in range(x_batch.shape[0]):
            x_batch_hog[i] = extract_hog_features(x_batch[i])
        yield x_batch_hog, y_batch

train_hog_generator = get_data_generator(train_generator)
val_hog_generator = get_data_generator(val_generator)

model = Sequential()
#model.add(Reshape((42, 42, 1), input_shape=(1764,)))
model.add(Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=(64, 64, 3))
)

model.add(Conv2D(32, (3, 3), padding='same', activation='relu'))
model.add(MaxPooling2D((2, 2)))

model.add(Conv2D(64, (3, 3), padding='same', activation='relu'))
model.add(Conv2D(64, (3, 3), padding='same', activation='relu'))
model.add(MaxPooling2D((2, 2)))

model.add(Flatten()) model.add(Dense(218,
activation='relu')) model.add(Dropout(0.2))
model.add(Dense(36, activation='softmax'))

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
from keras.callbacks import EarlyStopping, ReduceLROnPlateau
early_stop = EarlyStopping(monitor='val_loss', patience=15, verbose=1, mode='min')

reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.1, patience=5, verbose=1,
mode='min', min_lr=0.0001)

history = model.fit(train_generator, epochs=15, validation_data=val_generator, callbacks
=[early_stop, reduce_lr])

```

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.savefig("resenet50_with_augmentation.png")
plt.show()
```

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.savefig("resenet50_with_augmentation_loss.png")
plt.show()
```

Classification using SVM:

```
import os
import cv2
import numpy as np
from sklearn import svm
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score, recall_score, f1_score
from skimage.feature import hog
from skimage import exposure
Datadir = '/content/drive/MyDrive/datasplit'
categories =
['0','1','2','3','4','5','6','7','8','9','A','B','C','D','E','F','G','H','I','J','K','L','M','N','O','
P','Q','R','S','T','U','V','W','X','Y','Z']

train_data = []
for category in categories:
    path = os.path.join('/content/drive/MyDrive/datasplit/Train', category)
    label = categories.index(category)
    for img_name in os.listdir(path)
```

```

img_path = os.path.join(path, img_name)
img_array = cv2.imread(img_path)
img_array = cv2.resize(img_array, (100, 100))

fd, hog_image = hog(img_array, orientations=9, pixels_per_cell=(8, 8), cells_per_block=(2, 2), visualize=True, multichannel=True)
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))
train_data.append([fd, label])

X_train = [] y_train = []
for features, label in train_data: X_train.append(features) y_train.append(label)

test_data = []
for category in categories:
    path = os.path.join('/content/drive/MyDrive/datasplit/Test', category) label = categories.index(category)
    for img_name in os.listdir(path):
        img_path = os.path.join(path, img_name) img_array = cv2.imread(img_path)
        img_array = cv2.resize(img_array, (100, 100))
        fd, hog_image = hog(img_array, orientations=9, pixels_per_cell=(8, 8), cells_per_block=(2, 2), visualize=True, multichannel=True)
        hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))
        test_data.append([fd, label])

X_test = [] y_test = []
for features, label in test_data: X_test.append(features) y_test.append(label)

clf = svm.SVC(kernel='linear', C=1, random_state=42) clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))

```

Real time testing:

```
import tensorflow as tf
from tensorflow import keras
from keras.models import Sequential
from keras.layers import Activation, Dense, Flatten, BatchNormalization, Conv2,
MaxPool2D, Dropout
from keras.optimizers import Adam, SGD
from keras.metrics import categorical_crossentropy
from keras.preprocessing.image import ImageDataGenerator
```

```
import warnings
import numpy as np
import cv2
from keras.callbacks import ReduceLROnPlateau
from keras.callbacks import ModelCheckpoint, EarlyStopping
warnings.simplefilter(action='ignore', category=FutureWarning)
```

```
background = None
accumulated_weight = 0.5
```

```
ROI_top = 100
ROI_bottom = 300
ROI_right = 150
ROI_left = 350
```

```
def cal_accum_avg(frame, accumulated_weight):
```

```
    global background
```

```
    if background is None:
```

```
        background = frame.copy().astype("float")
```

```
        return None
```

```
    cv2.accumulateWeighted(frame, background, accumulated_weight)
```

```
def segment_hand(frame, threshold=25):
```

```
    global background
```

```

diff = cv2.absdiff(background.astype("uint8"), frame)

_, thresholded = cv2.threshold(diff, threshold, 255, cv2.THRESH_BINARY)

cv2.findContours(thresholded.copy(), cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

if len(contours) == 0:
    return None
else:

    hand_segment_max_cont = max(contours, key=cv2.contourArea)

    return (thresholded, hand_segment_max_cont)

cam = cv2.VideoCapture(0)

num_frames = 0
element = 10
num_imgs_taken = 0

while True:
    ret, frame = cam.read()
    frame = cv2.flip(frame, 1)

    frame_copy = frame.copy()

    roi = frame[ROI_top:ROI_bottom, ROI_right:ROI_left]

    gray_frame = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    gray_frame = cv2.GaussianBlur(gray_frame, (9, 9), 0)

    if num_frames < 60:
        cal_accum_avg(gray_frame, accumulated_weight)
        if num_frames <= 59:

            cv2.putText(frame_copy, "FETCHING BACKGROUND...PLEASE WAIT",
(80, 400), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0,0,255), 2)

    elif num_frames <= 300:

```

```

hand = segment_hand(gray_frame)

cv2.putText(frame_copy, "Adjust hand...Gesture for" + str(element), (200, 400),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255),2)

if hand is not None:

    thresholded, hand_segment = hand
    cv2.drawContours(frame_copy, [hand_segment + (ROI_right, ROI_top)], -1,
(255, 0, 0),1)
    cv2.putText(frame_copy, str(num_frames)+"For" + str(element), (70, 45),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)
    cv2.imshow("Thresholded Hand Image", thresholded)

else:

    hand = segment_hand(gray_frame)

    if hand is not None:

        thresholded, hand_segment = hand

        cv2.drawContours(frame_copy, [hand_segment + (ROI_right, ROI_top)], -1,(255,
0, 0),1)

        cv2.putText(frame_copy, str(num_frames), (70, 45),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

        cv2.putText(frame_copy, str(num_imgs_taken) + 'images' + "For" + str(element),
(200, 400), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

        cv2.imshow("Thresholded Hand Image", thresholded)
        if num_imgs_taken <= 300:
            cv2.imwrite(r"D:\\gesture\\train\\"+str(element)+"\\" +
str(num_imgs_taken+300) + '.jpg', thresholded)

        else:
            break
        num_imgs_taken +=1
    else:
        cv2.putText(frame_copy, 'No hand detected...!', (200, 400),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

```

```
cv2.rectangle(frame_copy, (ROI_left, ROI_top), (ROI_right,ROI_bottom),
(255,128,0), 3)
```

```
cv2.putText(frame_copy, "DataFlair hand sign recognition___", (10, 20),
cv2.FONT_ITALIC, 0.5, (51,255,51), 1)
```

```
num_frames += 1
cv2.imshow("Sign Detection", frame_copy)
k = cv2.waitKey(1) & 0xFF
```

```
if k == 27:
    break
```

```
cv2.destroyAllWindows()
cam.release()
```

```
img, labels = next(train_datagen )
```

```
def plotImages(images_arr):
    fig, axes = plt.subplots(1, 10, figsize=(30,20))
    axes = axes.flatten()
    for img, ax in zip( images_arr, axes):
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        ax.imshow(img)
        ax.axis('off')
    plt.tight_layout()
    plt.show()
```

```
plotImages(img)
print(img.shape)
print(labels)
```

```
img, labels = next(test_datagen)
```

```
scores = model.evaluate(img, labels, verbose=0)
print(f'{model.metrics_names[0]} of {scores[0]}; {model.metrics_names[1]} of
{scores[1]*100}%')
```

```
model.save() function.
model.save('model.h5')
```

```

word_dict =
{0:'One',1:'Ten',2:'Two',3:'Three',4:'Four',5:'Five',6:'Six',7:'Seven',8:'Eight',9:'Nine'}
predictions = model.predict(imgs, verbose=0)
print("predictions on a small set of test data--")
print("")
for ind, i in enumerate(predictions):
    print(word_dict[np.argmax(i)], end=' ')

plotImages(imgs)
print('Actual labels')
for i in labels:
    print(word_dict[np.argmax(i)], end=' ')
model = keras.models.load_model(r"C:\Users\abhi\best_model_dataflair3.h5")

background = None
accumulated_weight = 0.5

ROI_top = 100
ROI_bottom = 300
ROI_right = 150
ROI_left = 350
def cal_accum_avg(frame, accumulated_weight):
    global background
    if background is None:
        background = frame.copy().astype("float")
        return None

    cv2.accumulateWeighted(frame, background, accumulated_weight)

def segment_hand(frame, threshold=25):
    global background
    diff = cv2.absdiff(background.astype("uint8"), frame)
    __, thresholded = cv2.threshold(diff, threshold, 255, cv2.THRESH_BINARY)
    image, contours, hierarchy = cv2.findContours(thresholded.copy(),
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    if len(contours) == 0:
        return None
    else:
        hand_segment_max_cont = max(contours, key=cv2.contourArea)
        return (thresholded, hand_segment_max_cont)

cam = cv2.VideoCapture(0)

```

```

num_frames = 0
while True:
    ret, frame = cam.read()
    frame = cv2.flip(frame, 1)
    frame_copy = frame.copy()
    roi = frame[ROI_top:ROI_bottom, ROI_right:ROI_left]
    gray_frame = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    gray_frame = cv2.GaussianBlur(gray_frame, (9, 9), 0)

    if num_frames < 70:
        cal_accum_avg(gray_frame, accumulated_weight)
        cv2.putText(frame_copy, "FETCHING BACKGROUND...PLEASE WAIT",
(80, 400), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0,0,255), 2)

    else:
        hand = segment_hand(gray_frame)
        if hand is not None:
            thresholded, hand_segment = hand
            cv2.drawContours(frame_copy, [hand_segment + (ROI_right,
ROI_top)], -1, (255, 0, 0), 1)
            cv2.imshow("Thresholded Hand Image", thresholded)
            thresholded = cv2.resize(thresholded, (64, 64))
            thresholded = cv2.cvtColor(thresholded, cv2.COLOR_GRAY2RGB)
            thresholded = np.reshape(thresholded,
(1,thresholded.shape[0],thresholded.shape[1],3))

            pred = model.predict(thresholded)
            cv2.putText(frame_copy, word_dict[np.argmax(pred)], (170, 45),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

            cv2.rectangle(frame_copy, (ROI_left, ROI_top), (ROI_right, ROI_bottom),
(255,128,0), 3)

        num_frames += 1
        cv2.putText(frame_copy, "DataFlair hand sign recognition __",
(10, 20), cv2.FONT_ITALIC, 0.5, (51,255,51), 1)
        cv2.imshow("Sign Detection", frame_copy)
        k = cv2.waitKey(1) & 0xFF

        if k == 27:
            break

cam.release()
cv2.destroyAllWindows()

```

CHAPTER 4

SNAPSHOTS

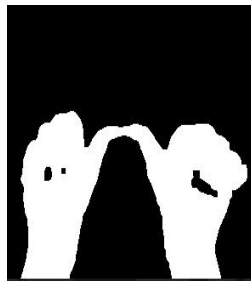
OUTPUT FOR PREPROCESSED IMAGES:

Alphabets:

Sign-A



Original



Mask



Binary image



Canny edge detection

Sign-C



Original



Mask



Binary image



Canny edge detection

Digits:

Sign-4



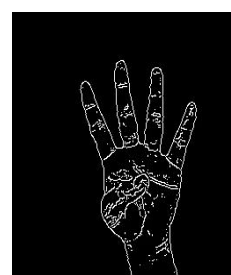
Original



Mask



Binary image

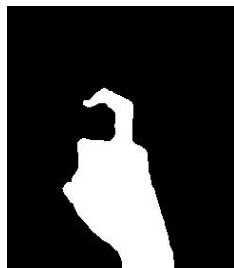


canny edge detection

Sign-7



Original



Mask



Binary image



canny edge detection

Fig 4.1 Preprocessed Images

OUTPUT FOR FEATURE EXTRACTION USING ORB:

Sign-5



Sign-7



Sign-A



Sign-C

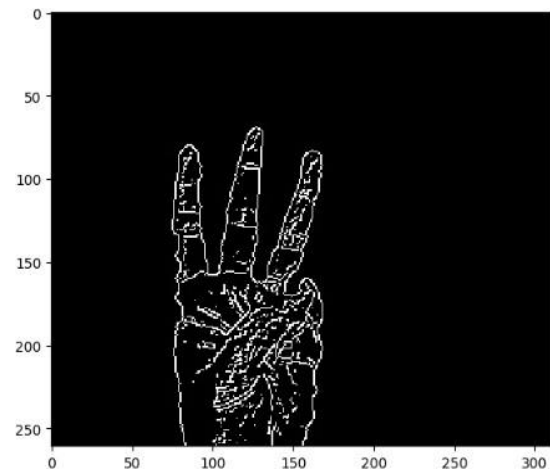


Fig 4.2 Feature Extraction Using ORB

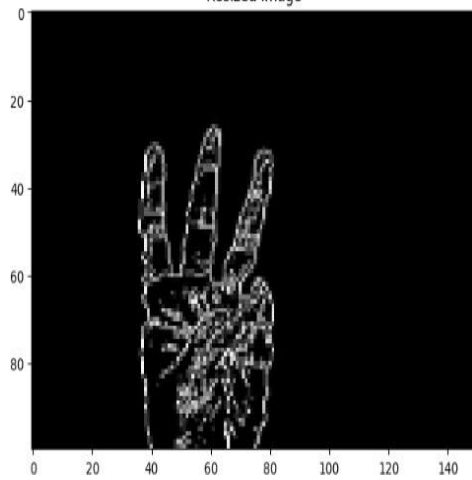
OUTPUT FOR FEATURE EXTRACTION USING HOG:

Sign-3

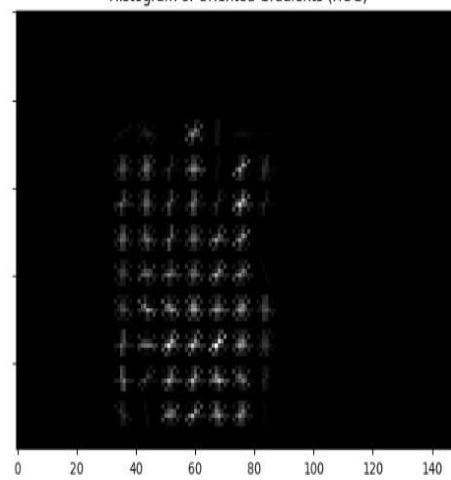
Original image



Resized image



Histogram of Oriented Gradients (HOG)



Sign-C

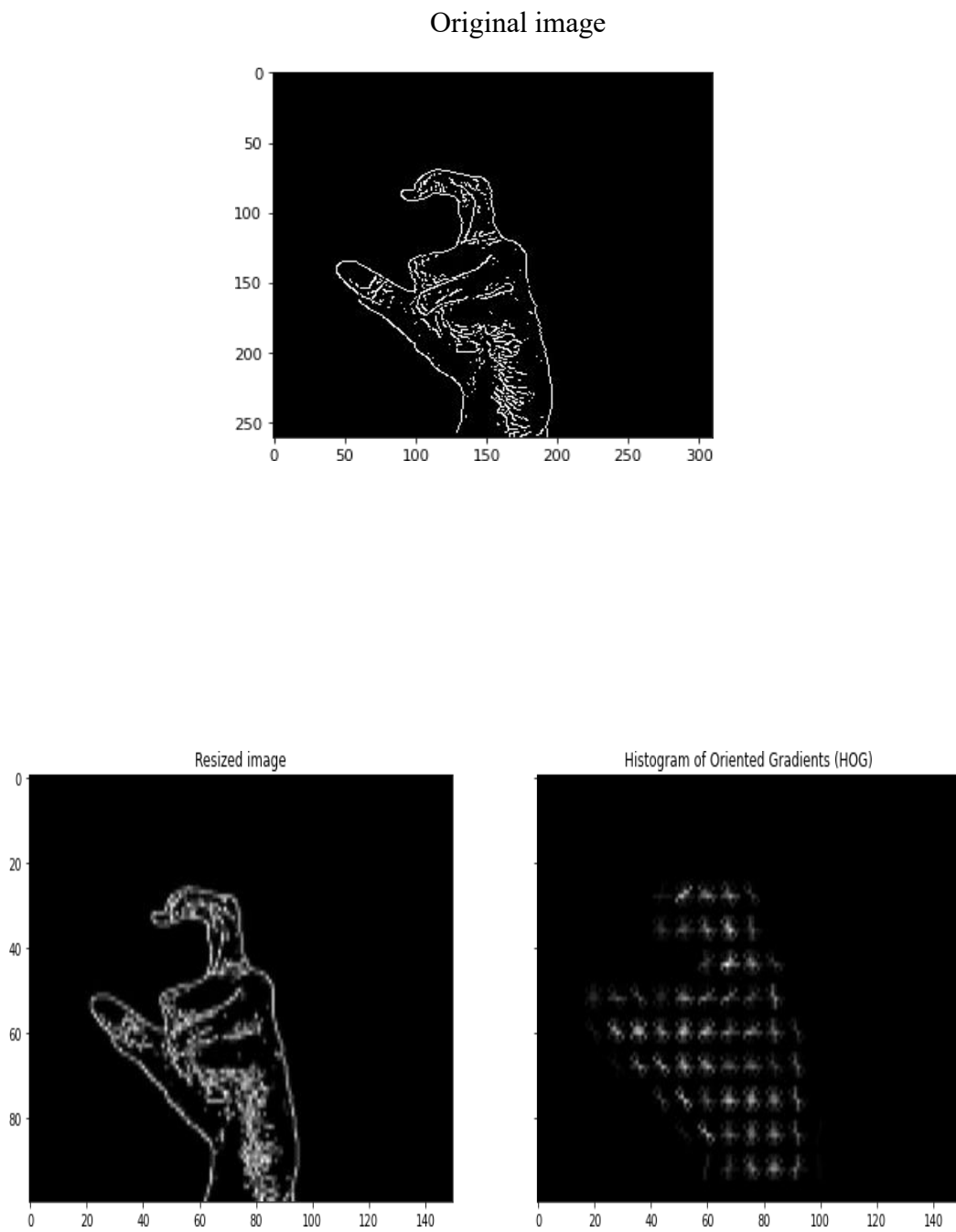


Fig 4.3 Feature Extraction Using HOG

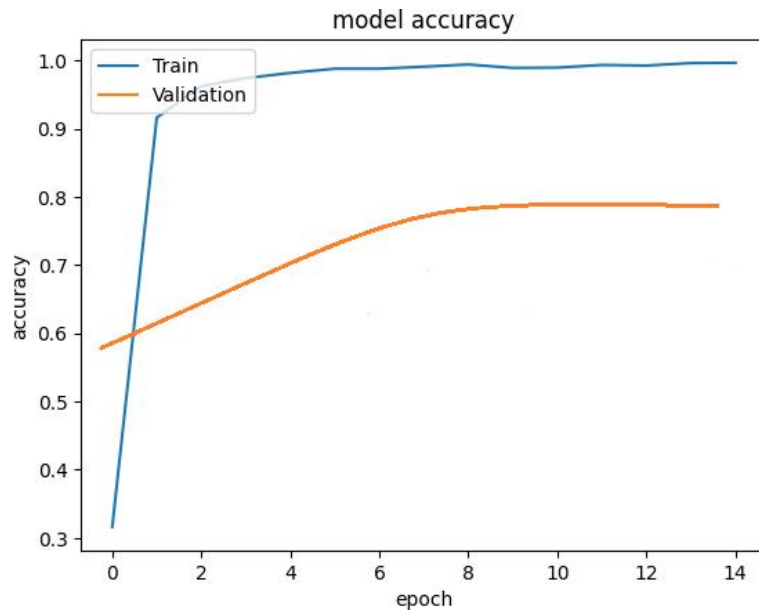
OUTPUT FOR CLASSIFICATION USING CNN:

```
Epoch 1/15
137/137 [=====] - 154s 1s/step - loss: 2.4897 - accuracy: 0.3160 - val_loss: 2.0432 - val_accuracy: 0.5875 - lr: 0.0010
Epoch 2/15
137/137 [=====] - 147s 1s/step - loss: 0.3085 - accuracy: 0.9162 - val_loss: 2.1344 - val_accuracy: 0.6074 - lr: 0.0010
Epoch 3/15
137/137 [=====] - 147s 1s/step - loss: 0.1288 - accuracy: 0.9621 - val_loss: 2.0726 - val_accuracy: 0.5586 - lr: 0.0010
Epoch 4/15
137/137 [=====] - 143s 1s/step - loss: 0.0859 - accuracy: 0.9741 - val_loss: 1.9669 - val_accuracy: 0.6519 - lr: 0.0010
Epoch 5/15
137/137 [=====] - 145s 1s/step - loss: 0.0560 - accuracy: 0.9816 - val_loss: 1.9048 - val_accuracy: 0.6830 - lr: 0.0010
Epoch 6/15
137/137 [=====] - 144s 1s/step - loss: 0.0391 - accuracy: 0.9878 - val_loss: 2.3059 - val_accuracy: 0.6385 - lr: 0.0010
Epoch 7/15
137/137 [=====] - 147s 1s/step - loss: 0.0423 - accuracy: 0.9878 - val_loss: 2.1215 - val_accuracy: 0.6230 - lr: 0.0010
Epoch 8/15
137/137 [=====] - 149s 1s/step - loss: 0.0301 - accuracy: 0.9908 - val_loss: 1.8286 - val_accuracy: 0.6941 - lr: 0.0010
Epoch 9/15
137/137 [=====] - 147s 1s/step - loss: 0.0221 - accuracy: 0.9940 - val_loss: 2.5329 - val_accuracy: 0.6891 - lr: 0.0010
Epoch 10/15
137/137 [=====] - 146s 1s/step - loss: 0.0344 - accuracy: 0.9890 - val_loss: 2.2272 - val_accuracy: 0.6163 - lr: 0.0010
Epoch 11/15
137/137 [=====] - 148s 1s/step - loss: 0.0324 - accuracy: 0.9894 - val_loss: 2.3879 - val_accuracy: 0.6474 - lr: 0.0010
Epoch 12/15
137/137 [=====] - 144s 1s/step - loss: 0.0240 - accuracy: 0.9933 - val_loss: 3.2106 - val_accuracy: 0.6663 - lr: 0.0010
Epoch 13/15
137/137 [=====] - ETA: 0s - loss: 0.0218 - accuracy: 0.9924
Epoch 13: ReduceLROnPlateau reducing learning rate to 0.00010000000474974513.
137/137 [=====] - 147s 1s/step - loss: 0.0218 - accuracy: 0.9924 - val_loss: 2.7545 - val_accuracy: 0.6502 - lr: 0.0010
Epoch 14/15
137/137 [=====] - 147s 1s/step - loss: 0.0145 - accuracy: 0.9961 - val_loss: 2.8908 - val_accuracy: 0.7029 - lr: 1.0000e-04
Epoch 15/15
137/137 [=====] - 149s 1s/step - loss: 0.0115 - accuracy: 0.9966 - val_loss: 2.8828 - val_accuracy: 0.6991 - lr: 1.0000e-04
```

Fig 4.4 Classification Using CNN

MODEL GRAPH:

ACCURACY GRAPH OF CNN



LOSS GRAPH OF CNN

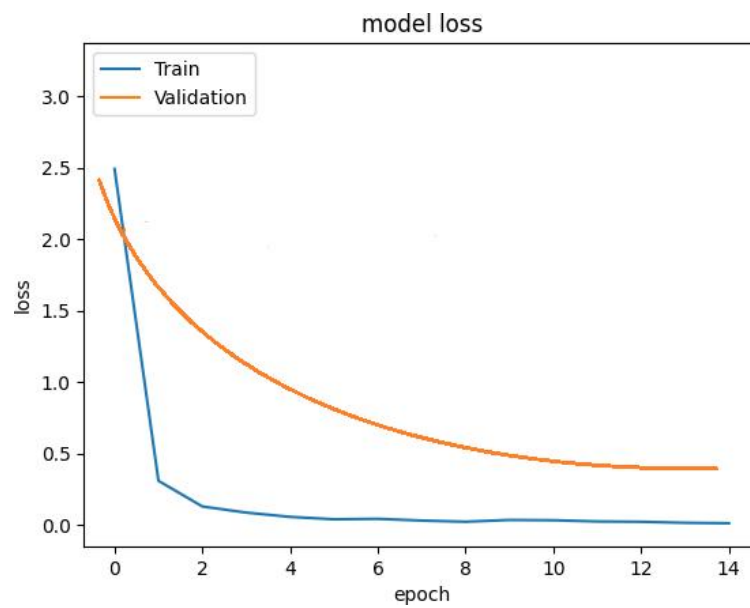


Fig 4.5 Model Graphs of CNN

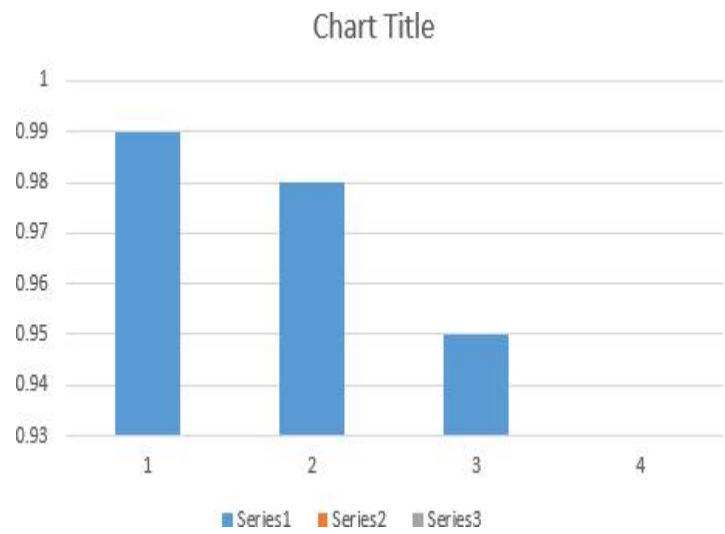
OUTPUT FOR CLASSIFICATION USING SVM:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	50
1	0.62	1.00	0.76	50
2	0.00	0.00	0.00	50
3	0.48	0.92	0.63	50
4	0.00	0.00	0.00	50
5	1.00	0.40	0.57	50
6	1.00	0.16	0.28	50
7	0.33	1.00	0.50	50
8	0.62	1.00	0.77	50
9	0.69	1.00	0.82	50
10	1.00	1.00	1.00	50
11	1.00	0.04	0.08	50
12	0.00	0.00	0.00	51
13	0.00	0.00	0.00	50
14	0.30	0.96	0.45	50
15	1.00	0.32	0.48	50
16	0.72	0.94	0.82	50
17	1.00	0.56	0.72	50
18	1.00	0.24	0.39	50
19	0.00	0.00	0.00	50
20	1.00	1.00	1.00	50
21	1.00	0.52	0.68	50
22	0.40	1.00	0.57	50
23	1.00	0.78	0.88	50
24	0.96	1.00	0.98	50
25	1.00	0.52	0.68	50
26	0.00	0.00	0.00	50
27	0.83	1.00	0.91	50
28	0.92	0.68	0.78	50
29	1.00	1.00	1.00	50
30	1.00	0.68	0.81	50
31	0.93	1.00	0.96	50
32	1.00	1.00	1.00	50
33	1.00	1.00	1.00	50
34	1.00	1.00	1.00	50
35	1.00	1.00	1.00	50
accuracy			0.63	1801
macro avg	0.69	0.63	0.60	1801
weighted avg	0.69	0.63	0.60	1801

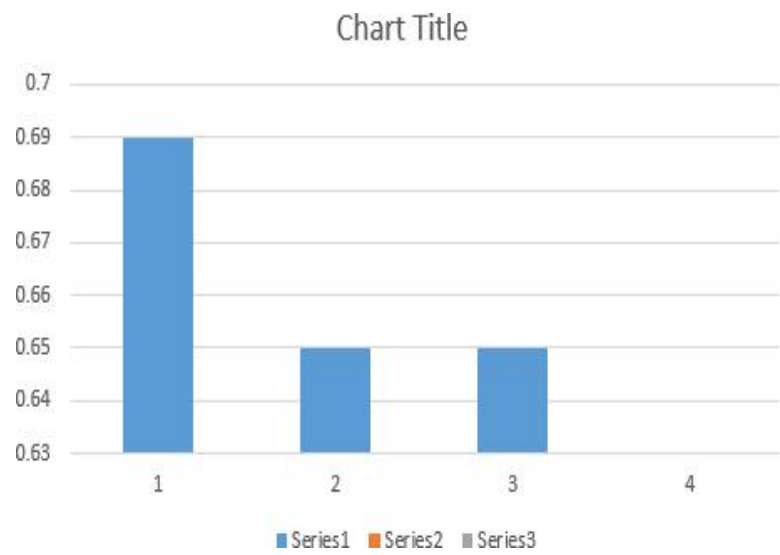
Fig 4.6 Classification Using SVM

ANALYSIS:

CNN:



SVM:



CHAPTER 5

CONCLUSION AND FUTURE PLANS

In this paper we have presented a technique to recognize and classify Indian sign language (A-Z and 0-9) using CNN and SVM. The objective of this work is to provide a more reliable and efficient means of communication so that it can be used anywhere. The dataset used here is a custom dataset, which helps to reduce the background dependency problem. The system is trained on all Indian sign language signs (26 alphabets and 10 digits) with an accuracy of 99%. In future, the dataset can be expanded by adding more signs from different languages of various countries, thereby achieving a more effective framework. The integration of multiple modalities, such as video and audio can be explored to improve the recognition of Indian sign language. A mobile application can be developed that can recognize ISL gestures on smartphones and other mobile devices.

CHAPTER 6

REFERENCES

1. Shukla, R., & Patel, R. (2016). Real-time sign language recognition using SURF and SVM. *International Journal of Engineering Research and General Science*, 4(2), 59-64.
2. Singh, S., Singh, J., & Verma, P. (2019). Sign language recognition using SURF feature extraction and SVM classifier. In *Proceedings of the 4th International Conference on Internet of Things and Connected Technologies (ICIoTCT 2019)* (pp. 173-177). IEEE.
3. Mehta, D., Vadera, R., & Singh, B. (2019). Indian Sign Language Recognition using SURF Features and HMM. In *Proceedings of the 2nd International Conference on Intelligent Sustainable Systems (ICISS 2019)* (pp. 175-181). Springer.
4. Sharma, S. K., & Kaur, R. (2019). Indian sign language recognition using SURF feature extraction and SVM classifier. *International Journal of Advanced Research in Computer Science*, 10(4), 119-122.
5. Sudhir S. Kanade, Padmanabh D. Deshpande Recognition of Indian sign language using SVM classifier *International Journal of Scientific Research and Development*, 2 (3) (2018).
6. G. Jayadeep, N.V. Vishnupriya, V. Venugopal, S. Vishnu, M. Geetha Mudra: convolutional neural network based Indian sign language translator for banks 4th International Conference on Intelligent Computing and Control Systems (ICICCS), 2020 (2020), pp. 1228-1232.
7. S. Reshna, A. Sajeena, M. Jayaraju Recognition of static hand gestures of Indian sign language using CNN AIP Conf Proc, 2222 (30012) (2020).

